

Codex no *VS Code*

Como usar a extensão do Codex dentro do editor: instalar e fazer login, usar como contexto o código que já está aberto, escolher onde a tarefa roda, revisar as mudanças ao lado do código e configurar a extensão.

`openai.chatgpt`

Contexto do editor

Slash commands

Local · Worktree · Cloud

`config.toml`

O Codex *ao lado do código*

A extensão coloca o agente do Codex numa barra lateral do editor. A vantagem sobre o app e a web é que **o contexto já está aberto**: os arquivos, a seleção e os chats recentes entram no prompt sem você precisar explicar tudo de novo.

CONTEXTO

Use o que está aberto

Referencie arquivos abertos, o trecho selecionado e chats recentes direto no composer. O Codex começa pelo código que você já está vendo.

REVISÃO

Mudanças ao lado do código

Leia o resumo e o diff focado, mantenha só as edições que quiser e peça ajustes no mesmo chat.

DELEGAÇÃO

Quando a tarefa cresce

Ittere localmente no que é rápido e mande para a nuvem (Codex web) o que precisa de mais tempo. O resultado volta para o mesmo editor.

Use a extensão quando...

- **Você está fazendo edições focadas** e quer os arquivos e o Codex na mesma tela.
- **Você quer revisar mudanças no lugar**, aplicando edições junto ao código-fonte.
- **Você está aprendendo um código desconhecido** e quer perguntar sobre os arquivos e símbolos abertos.
- **Você quer delegar uma tarefa maior** para a nuvem sem sair do editor.

Instalando a *extensão*

A mesma extensão, `openai.chatgpt` no Marketplace, funciona no **VS Code**, **VS Code Insiders**, **Cursor** e **Windsurf**. Xcode e as IDEs JetBrains têm integrações próprias.

1

Instale

Procure por "Codex" na aba de extensões (`Ctrl` + `Shift` + `X`) ou use a linha de comando ao lado.

2

Abra o Codex

Clique no ícone do Codex. Se ele não aparecer, rode **Codex: Open Codex Sidebar** na Command Palette.

3

Faça login

Entre com a conta do ChatGPT. Com uma chave de API a extensão funciona, mas **sem os recursos de nuvem**.

4

Primeiro chat

Com o projeto aberto, peça para explicar o código, fazer uma mudança focada ou depurar um problema.

INSTALAÇÃO PELO TERMINAL

```
code --install-extension openai.chatgpt
```

Boa prática da documentação: crie checkpoints no Git (commit) antes e depois de cada tarefa. Assim fica fácil reverter o que o agente mudou.

Windows: se os seus repositórios estão no WSL2, ative `chatgpt.runCodexInWindowsSubsystemForLinux` . O agente passa a rodar no WSL2, com o sandbox do Linux. No **Linux**, o sandbox precisa do `bubblewrap` , como vimos na aula 2.

Usando o contexto do *editor*

O grande diferencial da extensão é passar o código certo para o agente com poucos cliques.

COMO	O QUE FAZ
Contexto automático	Arquivos abertos e a seleção atual acompanham o prompt. Ligue ou desligue com <code>/ide-context</code> .
<code>chatgpt.addToThread</code>	Adiciona o trecho selecionado como contexto do chat atual.
<code>chatgpt.addFileToThread</code>	Adiciona o arquivo inteiro como contexto do chat atual.
CodeLens em TODO	Mostra um atalho acima dos comentários <code>TODO</code> para o Codex resolvê-los.

EXEMPLO DE PROMPT COM A SELEÇÃO ATIVA

`#` com a função `parse_date()` selecionada no editor
Explique o que este trecho faz e por que ele falha com datas no formato "28/09/2026".
Corrija sem mudar a assinatura da função e adicione um teste para o caso.

Com a seleção como contexto, você não precisa colar o código nem dizer o caminho do arquivo. Foque o prompt no **resultado esperado** e em **como validar**.

Local, Worktree ou *Cloud* pelo editor

Os mesmos três lugares de execução do app (aula 2) estão disponíveis na extensão. Você troca com slash commands digitados no composer.

/LOCAL

Workspace local

Roda na pasta aberta no editor. É o modo para iteração rápida e prática.

/WORKTREE

Nova worktree Git

Roda numa cópia isolada do repositório, sem mexer nos arquivos que você está editando.

/CLOUD

Nuvem

Envia o chat para o Codex web, quando a execução em nuvem está disponível. Ideal para tarefas longas.

/CLOUD-ENVIRONMENT

Escolher o environment

Define qual environment da nuvem (aula 1) o chat vai usar.

PERMISSÕES

O controle fica **abaixo do composer**, como no app. O sandbox define até onde o agente vai sozinho, e as aprovações definem quando ele para e pede permissão. O comando `/approve` aprova uma nova tentativa depois de uma recusa do revisor automático.

A extensão e a CLI usam o **mesmo agente e a mesma configuração**. Um chat em nuvem iniciado no editor aparece no Codex web e pode ser revisado lá, e vice-versa.

Slash commands *essenciais*

Digite `/` no composer, filtre (ex.: `/status`) e pressione `Enter`. Os principais:

COMANDO	PARA QUÊ
<code>/init</code>	Gera um <code>AGENTS.md</code> inicial para o projeto
<code>/plan</code>	Liga ou desliga o modo de planejamento em várias etapas
<code>/goal</code>	Define um objetivo persistente para o Codex perseguir
<code>/review</code>	Code review das mudanças não commitadas ou em relação a um branch base
<code>/model</code> · <code>/reasoning</code>	Escolhe o modelo e o esforço de raciocínio do chat

COMANDO	PARA QUÊ
<code>/status</code>	Mostra o ID do chat, o uso de contexto e os limites de uso
<code>/compact</code>	Compacta o contexto do chat atual
<code>/side</code>	Abre um chat lateral temporário sem interromper o principal
<code>/fork</code>	Copia um chat local para um novo chat local
<code>/mcp</code>	Mostra o status dos servidores MCP conectados

Paralelo com o Claude Code: o `AGENTS.md` faz o papel do `CLAUDE.md`, e o `/init` e o `/compact` funcionam de forma parecida nas duas ferramentas.

Revisando o trabalho *no editor*

Ao terminar, o Codex mostra um resumo e as linhas alteradas ali mesmo, sem painel extra. Você mantém o que quiser e pede ajustes no mesmo chat.

1 • DIFF FOCADO

Inspecione

Veja só os arquivos afetados, com a justificativa do agente ao lado do código.

2 • SELECIONE

Mantenha o que quiser

Aceite as edições que fazem sentido e descarte o resto antes de seguir.

3 • /REVIEW

Segunda opinião

Rode o code review nas mudanças não commitadas ou contra um branch base antes de commitar.

MENSAGENS DURANTE A EXECUÇÃO

Por padrão (`queue`), o que você envia enquanto o Codex trabalha **entra na fila** para a próxima rodada. No modo `steer`, a mensagem **redireciona a rodada atual**. `Ctrl / Cmd` + `Shift` + `Enter` inverte o comportamento só para aquela mensagem.

O `/review` roda **no chat atual** quando possível (`inline`). Para abrir a revisão num chat separado, mude `chatgpt.reviewDelivery` para `detached`.

As duas camadas de *configuração*

A extensão separa o **comportamento do agente** (compartilhado com a CLI) do **comportamento do editor**.

CODEx SETTINGS · CONFIG.TOML

Modelo, esforço de raciocínio, permissões, sandbox, servidores MCP e personalização. Abra pela engrenagem da barra lateral em **Codex Settings** → **Open config.toml**.

```
# pessoal (vale para CLI e extensão)
~/.codex/config.toml

# por projeto (só em projetos confiáveis)
.codex/config.toml
```

Ordem de precedência: flags da CLI, depois o `.codex/config.toml` do projeto e por último o pessoal. Vale a regra mais específica.

EDITOR SETTINGS (CHATGPT.*)	PADRÃO
<code>followUpQueueMode</code>	<code>queue</code> (ou <code>steer</code>)
<code>composerEnterBehavior</code>	<code>enter</code> envia; opções para exigir <code>Ctrl</code> / <code>Cmd</code> + <code>Enter</code>
<code>reviewDelivery</code>	<code>inline</code> (ou <code>detached</code>)
<code>commentCodeLensEnabled</code>	<code>true</code> : CodeLens nos TODOs
<code>openOnStartup</code>	<code>false</code> : foca a barra do Codex ao iniciar
<code>localeOverride</code>	Auto: idioma da interface do Codex
<code>runCodexInWindowsSubsystemForLinux</code>	<code>false</code> : só no Windows, para rodar no WSL2

Nas configurações do editor, filtre por `@ext:openai.chatgpt`.

Comandos e *atalhos de teclado*

Todos os comandos da extensão ficam na Command Palette (`Ctrl` + `Shift` + `P` , ou `Cmd` + `Shift` + `P` no macOS), e qualquer um pode ganhar um atalho próprio.

COMANDO	ATALHO PADRÃO	FUNÇÃO
<code>chatgpt.newChat</code>	<code>Ctrl</code> + <code>N</code> · <code>Cmd</code> + <code>N</code>	Novo chat
<code>chatgpt.openSidebar</code>	—	Abre a barra lateral
<code>chatgpt.newCodexPanel</code>	—	Novo painel do Codex
<code>chatgpt.openCommandMenu</code>	—	Menu de comandos do Codex
<code>chatgpt.addToThread</code>	—	Seleção → contexto
<code>chatgpt.addFileToThread</code>	—	Arquivo → contexto

CRIAR UM ATALHO

1. Abra a Command Palette.
2. Rode **Preferences: Open Keyboard Shortcuts**.
3. Busque `Codex` ou o ID (ex.: `chatgpt.addToThread`).
4. Clique no lápis e grave a combinação.

Sugestão: dê um atalho ao `chatgpt.addToThread` . Selecionar código e mandar para o chat vira um gesto só.

Teste *rápido*

1. Qual é o ID da extensão oficial do Codex no Marketplace do VS Code?

A) openai.codex

B) openai.chatgpt

C) github.copilot

D) openai.codex-cli

B. A extensão é a `openai.chatgpt`, a mesma no VS Code, Cursor e Windsurf.

2. Você está no meio de uma edição e quer delegar uma tarefa longa sem ocupar a máquina. Qual comando usar?

A) `/local`

B) `/compact`

C) `/cloud`

D) `/fork`

C. O `/cloud` envia o chat para o Codex web, usando o environment escolhido com `/cloud-environment`.

3. Onde ficam o modelo, as permissões e os servidores MCP que a extensão compartilha com a CLI?

A) settings.json do VS Code

B) config.toml

C) AGENTS.md

D) package.json

B. O comportamento do agente fica no `config.toml` (`~/.codex/` ou `.codex/` do projeto). As configurações `chatgpt.*` cuidam só do editor.

4. Com `followUpQueueMode` no padrão, o que acontece com uma mensagem enviada enquanto o Codex trabalha?

A) Interrompe e descarta a rodada

B) Entra na fila para a próxima rodada

C) É ignorada

D) Abre um novo chat

B. O padrão é `queue`. Com `steer` a mensagem redireciona a rodada atual, e `Ctrl / Cmd + Shift + Enter` inverte o comportamento uma vez.

○ RECAPITULANDO

O agente onde *o código já está*

A extensão `openai.chatgpt` traz o Codex para o VS Code, o Cursor e o Windsurf. Ela usa como contexto os arquivos e a seleção abertos, alterna entre local, worktree e nuvem com slash commands, mostra o diff ao lado do código e compartilha o `config.toml` com a CLI.

`openai.chatgpt``/ide-context``/local · /worktree · /cloud``/review``config.toml`

Próxima aula → A interface via CLI

Fontes: learn.chatgpt.com/docs (codex/ide · developer-commands?surface=ide · developer-settings?surface=ide · config-file/config-basic · sandboxing · windows) e marketplace.visualstudio.com/items?itemName=openai.chatgpt. Interface, comandos e disponibilidade podem mudar; confira a documentação oficial.